

Rapport détaillé de réalisation

Module NoSQL Security — Séance 2

Modélisation NoSQL, requêtes avancées, index et sécurité data-level

Objet du document. Ce rapport reconstruit le travail effectivement réalisé à partir de trois sources de preuve : le support de séance en HTML, le journal complet des commandes exécutées dans `mongosh`, et l'archive de captures d'écran. L'objectif est de produire un livrable exploitable tel quel, rédigé en $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ / `LuaLaTeX`, propre à l'impression et suffisamment détaillé pour documenter l'exécution des travaux pratiques et du devoir maison.

Support pédagogique	seance 2.html
Journal d'exécution	Mongo-commands.txt
Preuves visuelles	Screenhsots.zip (64 captures)
Base de travail	soc_lite puis ids_alerts
Environnement visé	Docker + MongoDB 7 / mongosh
Réalisateur	Mohammed Zguioui

Table des matières

1	Résumé exécutif	3
2	Sources exploitées et méthode de reconstruction	3
2.1	Plan pédagogique reconstitué	3
3	Environnement et corpus manipulé	4
3.1	Collections effectivement utilisées	4
3.2	Modèle conceptuel retenu	4
4	Travail réalisé pendant la séance	4
4.1	TP1 — Construction du modèle SOC-lite	4
4.2	Correction des erreurs initiales et insertion des incidents	6
4.3	Vérifications de cohérence de fin de TP1	8
4.4	TP2 — Incidents, commentaires, références et relation many-to-many	8
4.5	TP3 — Requêtes guidées sur les logs et incidents	9
4.6	TP4 — Agrégation et lecture analytique du SOC-lite	9
4.7	TP5 — Index, explain() et comportement du planificateur	10
4.8	TP6 — Sur-collecte, PII et projections sûres	10
5	Exécution du devoir maison	11
5.1	Partie 1 — Insertion du mini dataset ids_alerts	11
5.2	Partie 2 — Requêtes du devoir et résultats observés	11
5.3	Partie 3 — Index proposés et justifiés	13
6	Checklist sécurité remplie à partir des preuves	13
7	Analyse critique et enseignements	14
7.1	1. Embed/reference : compréhension correcte du compromis	14
7.2	2. L'indexation a été mesurée, pas supposée	14
7.3	3. La sécurité data-level est comprise comme un problème d'exposition, pas seulement de stockage	15
8	Conclusion	15
A	Extraits représentatifs des commandes exécutées	15
A.1	Erreur initiale puis correction	15
A.2	Séquence d'indexation	16
A.3	Masquage des emails	16

A.4 Extrait du devoir maison	16
B Inventaire condensé des séquences du journal	16

1 Résumé exécutif

Le corpus d'exécution montre une progression cohérente sur l'ensemble de la séance : construction du modèle **SOC-lite**, insertion des collections **users**, **security_logs** et **incidents**, ajout de commentaires embedded, création d'un troisième incident référencé, exécution des requêtes guidées, production d'agrégations SOC, mesure d'impact des index avec **explain()**, puis mise en œuvre des premières mesures de sécurité data-level (masquage des emails et projections sûres).

Deux erreurs initiales ont été observées avant stabilisation du modèle : une **RangeError: Maximum call stack size exceeded** et une **BSONError: Cannot convert circular structure to BSON**. Dans les deux cas, la séquence gagnante a consisté à matérialiser les curseurs MongoDB au moyen de **toArray().map(...)** avant de les réinjecter dans les documents d'incident. Une fois cette correction appliquée, les insertions et vérifications se sont déroulées normalement.

Le devoir maison a également été largement exécuté : insertion du mini jeu de données **ids_alerts**, filtres par sévérité, tri chronologique, projections minimales, agrégations par **rule_id**, **src_ip** et **sensor**, sélection des alertes **block/drop**, et requête sur les alertes critiques issues du réseau interne. La seule partie qui n'apparaît pas comme effectivement exécutée dans le journal est la création explicite des index du devoir ; elle est donc traitée dans ce rapport comme une *proposition justifiée* et non comme une opération capturée.

Synthèse en une phrase. Les preuves montrent un travail complet sur les TPs 1 à 6, ainsi qu'une exécution opérationnelle de la partie requêtes du devoir maison, avec une bonne compréhension des notions d'embed/reference, d'agrégation, d'indexation et de minimisation des données.

2 Sources exploitées et méthode de reconstruction

Le présent document a été reconstitué à partir des éléments suivants :

- le support **seance 2.html**, qui définit les objectifs, les TPs, la checklist sécurité et le devoir maison ;
- le journal **Mongo-commands.txt**, qui constitue la source principale pour la chronologie des commandes, les résultats, les messages d'erreur et les sorties de **explain()** ;
- l'archive **Screenhots.zip**, utilisée comme preuve visuelle complémentaire pour illustrer les moments clés de l'exécution.

La reconstruction a suivi une logique d'audit : extraction des séquences **soc_lite>**, regroupement par TP, comparaison avec le plan pédagogique, puis consolidation des résultats en un récit technique continu. Au total, le journal brut contient 52 séquences de commandes distinctes et l'archive d'images contient 64 captures exploitables.

2.1 Plan pédagogique reconstitué

Le support de séance structure le travail en six TPs successifs :

1. TP1 — construction du modèle **SOC-lite** ;
2. TP2 — incidents, commentaires et références logs ;
3. TP3 — requêtes guidées ;
4. TP4 — statistiques par agrégation ;
5. TP5 — index et **explain()** ;

6. TP6 — sur-collecte, PII et projections sûres.

Le devoir maison ajoute un mini jeu de données `ids_alerts`, dix requêtes d'analyse et une proposition de deux index.

3 Environnement et corpus manipulé

Le support de séance prévoit un lancement via Docker avec `mongo:7` exposé sur le port 27017, puis une connexion par `mongosh`. Les sorties `explain()` du journal montrent un serveur MongoDB 7.0.30, cohérent avec cette consigne.

3.1 Collections effectivement utilisées

Collection	Rôle fonctionnel observé
<code>users</code>	Référentiel des utilisateurs SOC, équipes, rôles et données de contact.
<code>security_logs</code>	Journal brut d'événements sécurité horodatés, exploité à la fois pour la corrélation et l'agrégation.
<code>incidents</code>	Gestion des cas, avec commentaires embarqués et références vers des logs liés.
<code>ids_alerts</code>	Jeu de données du devoir maison dédié à des alertes IDS/IPS.

3.2 Modèle conceptuel retenu

Le fil conducteur du support est la plateforme `SOC-lite`. Le modèle mis en œuvre dans les preuves confirme les choix pédagogiques attendus :

- **embed** pour les tableaux de rôles et les commentaires d'incident ;
- **reference** pour l'association entre incidents et logs via `related_log_ids` ;
- **append-only** pour les logs de sécurité ;
- début de **many-to-many** avec le champ `assigned_analysts`.

4 Travail réalisé pendant la séance

4.1 TP1 — Construction du modèle `SOC-lite`

La première phase documentée correspond à l'amorçage du modèle. La collection `users` a été insérée avec quatre comptes (`alice_analyst`, `bob_senior`, `charlie_admin`, `diana_junior`), chacun portant un nom complet, une adresse email, une équipe, un tableau de rôles et des dates de création / de dernière connexion. Cette structure respecte le pattern *one-to-few* attendu pour les rôles.

```
soc_lite> db.users.insertMany([
  {
    username: "alice_analyst",
    full_name: "Alice Dupont",
    email: "alice@soc.local",
    roles: ["analyst", "viewer"],
    team: "Blue Team",
    created_at: new Date("2025-01-15"),
    last_login: new Date("2025-03-01T08:30:00Z"),
    is_active: true
  },
  {
    username: "bob_senior",
    full_name: "Bob Martin",
    email: "bob@soc.local",
    roles: ["senior_analyst", "incident_manager"],
    team: "Blue Team",
    created_at: new Date("2024-06-10"),
    last_login: new Date("2025-03-03T14:12:00Z"),
    is_active: true
  },
  {
    username: "charlie_admin",
    full_name: "Charlie Leroy",
    email: "charlie@soc.local",
    roles: ["admin"],
    team: "SOC Management",
    created_at: new Date("2024-01-05"),
    last_login: new Date("2025-02-28T18:00:00Z"),
    is_active: true
  },
  {
    username: "diana_junior",
    full_name: "Diana Nour",
    email: "diana@soc.local",
    roles: ["viewer"],
    team: "Blue Team",
    created_at: new Date("2025-02-01"),
    last_login: new Date("2025-03-03T09:00:00Z"),
    is_active: true
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69d0ab723f56d9fcc18563b1'),
    '1': ObjectId('69d0ab723f56d9fcc18563b2'),
    '2': ObjectId('69d0ab723f56d9fcc18563b3'),
    '3': ObjectId('69d0ab723f56d9fcc18563b4')
  }
}
```

FIGURE 1 – Preuve visuelle de l'insertion de la collection `users` et retour d'acquiescement MongoDB.

La collection `security_logs` a ensuite été alimentée avec un jeu de huit événements couvrant des cas réalistes : `login_failed`, `login_success`, `port_scan`, `malware_detected` et `data_exfiltration_suspect`. Les champs utilisés sont conformes au support : horodatage, IP source/destination, ports, protocole, utilisateur visé, équipement émetteur, sévérité et `raw_log`.

Une première tentative d'insertion des incidents a toutefois échoué. L'idée était correcte — récupérer les `ObjectId` des logs puis les injecter dans `related_log_ids` — mais la matérialisation directe par `.map(d => d._id)` dans le flux d'exécution a provoqué une erreur dans l'environnement `mongosh` observé.

4.2 Correction des erreurs initiales et insertion des incidents

Deux erreurs précoces apparaissent clairement dans le journal :

- `RangeError: Maximum call stack size exceeded` lors d'une première tentative d'insertion de `INC-2025-001` et `INC-2025-002`;
- `BSONError: Cannot convert circular structure to BSON` lors d'une première tentative de création de `INC-2025-003`.

Le correctif réellement appliqué a consisté à **matérialiser explicitement les curseurs** avant l'insertion, selon le schéma suivant :

```
let logIds = db.security_logs
  .find({ src_ip: "192.168.1.105" })
  .toArray()
  .map(d => d._id);

let malwareLogIds = db.security_logs
  .find({ event_type: { $in: ["malware_detected", "data_exfiltration_suspect"] } })
  .toArray()
  .map(d => d._id);
```

Après cette correction, les incidents `INC-2025-001` et `INC-2025-002` ont été insérés avec succès. Le premier document implémente proprement le couplage pédagogique attendu : commentaires embarqués d'un côté, références vers les logs SSH de l'autre.

```
soc_lite> let logIds = db.security_logs
| .find({ src_ip: "192.168.1.105" })
| .toArray()
| .map(d => d._id);
soc_lite> let logIds = db.security_logs
| .find({ src_ip: "192.168.1.105" })
| .toArray()
| .map(d => d._id);

let malwareLogIds = db.security_logs
  .find({
    event_type: { $in: ["malware_detected", "data_exfiltration_suspect"] }
  })
  .toArray()
  .map(d => d._id);

db.incidents.insertMany([
  {
    incident_id: "INC-2025-001",
    title: "Brute force SSH sur srv-web-01",
    description: "Multiples tentatives SSH échouées depuis 192.168.1.105 ciblant
root et admin.",
    severity: "high",
    status: "investigating",
    created_at: new Date("2025-03-03T10:30:00Z"),
    updated_at: new Date("2025-03-03T11:00:00Z"),
    assigned_to: "alice_analyst",
    tags: ["brute-force", "ssh", "external"],
    related_log_ids: logIds,
    comments: [
      {
        author: "alice_analyst",
        text: "3 tentatives en 2s depuis la même IP. Pattern brute-force confirmé
.",
        timestamp: new Date("2025-03-03T10:35:00Z")
      },
      {
        author: "bob_senior",
        text: "IP bloquée au firewall. Surveiller si rebond depuis un autre vecte
ur.",
        timestamp: new Date("2025-03-03T10:50:00Z")
      }
    ],
    mitre_tactics: ["TA0006"],
    mitre_techniques: ["T1110.001"]
  },
],
```

FIGURE 2 – Insertion réussie des incidents après matérialisation des curseurs avec `toArray().map(...)` - 1 .


```

{
  incident_id: "INC-2025-002",
  title: "Malware détecté sur endpoint-pc-042",
  description: "Trojan détecté dans /tmp/update.exe. Potentielle exfiltration e
n cours.",
  severity: "critical",
  status: "active",
  created_at: new Date("2025-03-03T14:30:00Z"),
  updated_at: new Date("2025-03-03T16:50:00Z"),
  assigned_to: "bob_senior",
  tags: ["malware", "exfiltration", "endpoint"],
  related_log_ids: malwareLogIds,
  comments: [
    {
      author: "bob_senior",
      text: "Isolement réseau du poste effectué. Analyse forensique en cours.",
      timestamp: new Date("2025-03-03T14:45:00Z")
    }
  ],
  mitre_tactics: ["TA0010", "TA0011"],
  mitre_techniques: ["T1041", "T1071.001"]
}
]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69d26d602eaf52a8bf8563c0'),
    '1': ObjectId('69d26d602eaf52a8bf8563c1')
  }
}

```

FIGURE 3 – Insertion réussie des incidents après matérialisation des curseurs avec `toArray().map(...)` - 2 .

4.3 Vérifications de cohérence de fin de TP1

Les contrôles de cohérence exécutés ensuite confirment l'état suivant du jeu de données :

Collection	Valeur attendue	Valeur observée
<code>users.countDocuments()</code>	4	4
<code>security_logs.countDocuments()</code>	8	8
<code>incidents.countDocuments()</code>	2	2

La lecture détaillée de INC-2025-001 montre également que le document contient bien les trois `related_log_ids` issus des tentatives SSH sur 192.168.1.105, ainsi que deux commentaires embarqués.

4.4 TP2 — Incidents, commentaires, références et relation many-to-many

Le TP2 a été effectivement réalisé dans le bon ordre logique. D'abord, un nouveau commentaire a été ajouté à INC-2025-001 via `$push`, ce qui illustre le pattern *embed* sur un nombre restreint d'objets enfants. La projection subséquente sur `comments` confirme la présence de trois commentaires, dont un commentaire ajouté par `diana_junior`.

Ensuite, trois nouveaux logs ont été insérés pour documenter une séquence autour du compte `charlie` : deux `login_failed` suivis d'un événement `privilege_escalation`. Une fois encore, la création de la liste `charlieLogIds` a été matérialisée avec `toArray().map(...)` avant insertion de l'incident INC-2025-003.

La vérification de cohérence a été menée avec la séquence suivante : récupération de l'incident

INC-2025-003, puis comptage des logs dont l'`ObjectId` appartient à `related_log_ids`. Le résultat retourné est 3, ce qui valide la cohérence entre le document incident et les événements sources.

Enfin, la dimension *many-to-many* a été esquissée par l'ajout d'un tableau `assigned_analysts` à INC-2025-002 contenant `bob_senior` et `alice_analyst`. Cette étape est importante car elle montre le passage d'un simple champ d'assignation à un modèle plus souple de co-traitement de l'incident.

4.5 TP3 — Requêtes guidées sur les logs et incidents

Huit requêtes documentées permettent de montrer la maîtrise des filtres, des fenêtres temporelles, du tri, de la pagination et des projections. Les observations principales sont résumées ci-dessous.

Requête exécutée	Résultat observé
Logs critiques	Trois événements critiques sont retournés : <code>malware_detected</code> , <code>data_exfiltration_suspect</code> et <code>privilege_escalation</code> .
Fenêtre 10h–12h triée	Cinq événements sont visibles entre 10 :00 et 12 :00, ce qui couvre les trois échecs SSH, un succès de connexion et un <code>port_scan</code> .
Projection minimale sur les <code>login_failed</code>	La requête ne retourne que <code>timestamp</code> , <code>src_ip</code> et <code>user_attempted</code> , conformément au principe de minimisation.
Trois logs les plus récents	Les trois résultats les plus récents sont les trois événements liés à <code>charlie</code> à 17 :00 et 17 :01.
Filtre logique <code>src_ip</code> ou sévérité critique	La requête mixte confirme la maîtrise des opérateurs <code>\$or</code> .
Incidents actifs / investigant avec commentaires	Les incidents INC-2025-001 et INC-2025-002 sont restitués avec uniquement les auteurs et textes de commentaires.

4.6 TP4 — Agrégation et lecture analytique du SOC-lite

L'étape d'agrégation est l'un des points forts du travail réalisé. Cinq pipelines ont été exécutés. Ils permettent non seulement de vérifier la syntaxe MongoDB, mais surtout de produire des indicateurs SOC directement exploitables.

```
soc_lite> db.security_logs.aggregate([
|   { $group: { _id: "$src_ip", count: { $sum: 1 } } },
|   { $sort: { count: -1 } },
|   { $limit: 5 }
| ])
[
|   { _id: '10.0.1.200', count: 4 },
|   { _id: '192.168.1.105', count: 3 },
|   { _id: '10.0.1.50', count: 1 },
|   { _id: '172.16.0.88', count: 1 },
|   { _id: '192.168.2.30', count: 1 }
| ]
```

FIGURE 4 – Agrégation sur `security_logs` : top des IP sources observées dans le jeu de données.

Les résultats les plus structurants sont les suivants :

- **Top IPs sources** : 10.0.1.200 apparaît 4 fois, devant 192.168.1.105 avec 3 occurrences.
- **Échecs de login par utilisateur** : `charlie` totalise 3 échecs, `root` 2, `admin` 1.

- **Répartition des sévérités** : 5 événements `medium`, 3 `critical`, puis 1 `low`, 1 `info` et 1 `high`.
- **Alertes high/critical par device** : les devices `ids-sensor-02`, `siem-console`, `endpoint-pc-042` et `firewall-main` apparaissent chacun une fois.
- **Événements par heure le 3 mars** : la tranche 10:00 concentre 4 événements, et la tranche 17:00 en concentre 3.

Du point de vue métier, ces agrégations montrent que l'étudiant ne s'est pas limité à des filtres simples : il a utilisé `$match`, `$group`, `$sort`, `$limit` et `$dateToString`, soit l'essentiel du pipeline présenté dans le support.

4.7 TP5 — Index, `explain()` et comportement du planificateur

La séquence d'indexation a été exécutée de manière pédagogique et complète : mesure avant index, création d'un index simple, nouvelle mesure, ajout d'un index composé, lecture de la liste des index, puis observation du comportement du planificateur.

Avant index. La requête `find({src_ip: "192.168.1.105"}).explain("executionStats")` retourne un plan COLLSCAN avec `totalDocsExamined = 11`.

Après création de `src_ip_1`. La même requête passe en IXSCAN, avec `totalKeysExamined = 3` et `totalDocsExamined = 3`.

```
soc_lite> db.security_logs.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { src_ip: 1 }, name: 'src_ip_1' },
  {
    v: 2,
    key: { timestamp: 1, src_ip: 1 },
    name: 'timestamp_1_src_ip_1'
  }
]
```

FIGURE 5 – Liste des index présents à l'issue du TP5.

Les index présents en fin de TP5 sont donc : `_id_`, `src_ip_1` et `timestamp_1_src_ip_1`.

Un point particulièrement intéressant apparaît ensuite : sur la requête combinant une plage temporelle et `src_ip`, le planificateur conserve l'index simple `src_ip_1` comme plan gagnant et rejette le composé dans ce jeu de données. Autrement dit, l'ajout d'un index composé *n'implique pas automatiquement* qu'il sera choisi. Ce résultat illustre bien le message du support sur la *prefix rule* et, plus largement, sur la nécessité de mesurer l'effet réel des index au lieu de supposer leur utilité.

4.8 TP6 — Sur-collecte, PII et projections sûres

Le TP6 a lui aussi été exécuté de manière concrète. Il commence par la lecture d'un événement `malware_detected`, ce qui sert d'étape d'identification des champs à sensibilité élevée, notamment `raw_log`.

Le traitement le plus important est le **masquage des emails** dans la collection `users`. Un pipeline `updateMany` a été utilisé pour créer un champ dérivé `email_masked` au format `a***@soc.local`.



```
soc_lite> db.users.find({}, { username: 1, email: 1, email_masked: 1, _id: 0 })
[
  {
    username: 'alice_analyst',
    email: 'alice@soc.local',
    email_masked: 'a***@soc.local'
  },
  {
    username: 'bob_senior',
    email: 'bob@soc.local',
    email_masked: 'b***@soc.local'
  },
  {
    username: 'charlie_admin',
    email: 'charlie@soc.local',
    email_masked: 'c***@soc.local'
  },
  {
    username: 'diana_junior',
    email: 'diana@soc.local',
    email_masked: 'd***@soc.local'
  }
]
```

FIGURE 6 – Contrôle visuel du masquage des emails dans `users`.

Les projections suivantes ont ensuite été exécutées :

- projection `viewer` sur `users` limitée à `username`, `team`, `roles` et `email_masked` ;
- exclusion explicite de `raw_log` lors de la consultation des logs critiques par un tableau de bord `viewer`.

Cette phase montre une compréhension correcte de la différence entre *conserver la donnée* et *contrôler son exposition*. On ne supprime pas l'email ni le `raw_log` ; on adapte ce qui est visible selon le rôle et l'usage.

5 Exécution du devoir maison

5.1 Partie 1 — Insertion du mini dataset `ids_alerts`

Le mini jeu de données du devoir a bien été inséré. Il contient cinq alertes IDS/IPS réparties entre trois signatures : `SID-1001` (trois occurrences), `SID-2050` (une occurrence) et `SID-3001` (une occurrence). Les sévérités observées sont `high`, `critical` et `medium`, avec des actions `alert`, `block` et `drop`.

5.2 Partie 2 — Requêtes du devoir et résultats observés

Le journal prouve l'exécution effective de l'ensemble des requêtes attendues pour la partie analyse. Le tableau suivant synthétise les résultats observés.

Requête			Résultat	Observation
Alertes	high	ou	4 documents	Trois alertes <code>SID-1001</code> de sévérité <code>high</code> et une alerte <code>SID-2050</code> de sévérité <code>critical</code> .
critical				

Requête	Résultat	Observation
Alertes SID-1001 triées par temps	3 documents	Horodatages 08 :00, 08 :05 et 11 :00.
Nombre d'alertes par rule_id	3 groupes	SID-1001 = 3, SID-2050 = 1, SID-3001 = 1.
Fenêtre 08h–10h, projection minimale	3 documents	Retour limité à timestamp, rule_name et src_ip.
Deux alertes les plus récentes	2 documents	11 :00 SID-1001 puis 10 :30 SID-3001.
Top IPs sources	3 groupes	203.0.113.15 = 3, 10.0.2.100 = 1, 10.0.0.10 = 1.
Actions block ou drop	2 documents	Une alerte bloquée DNS tunneling et une alerte drop SQL injection.
Nombre d'alertes par sensor	2 groupes	ids-dmz-01 = 3, ids-internal-01 = 2.
Alertes critiques internes (src_ip commençant par 10.)	1 document	L'alerte SID-2050 (DNS Tunneling Detected) satisfait la condition.
Alerte la plus ancienne	1 document	SID-1001 à 08 :00.

```
soc_lite> db.ids_alerts.aggregate([
|   {
|     $group: {
|       _id: "$rule_id",
|       count: { $sum: 1 }
|     }
|   },
|   { $sort: { count: -1 } }
| ])
|
| [
|   { _id: 'SID-1001', count: 3 },
|   { _id: 'SID-2050', count: 1 },
|   { _id: 'SID-3001', count: 1 }
| ]
```

FIGURE 7 – Agrégation du devoir maison : top des IP sources dans ids_alerts.

```

soc_lite> db.ids_alerts.find(
| {
|   severity: "critical",
|   $expr: {
|     $eq: [
|       { $substrBytes: ["$src_ip", 0, 3] },
|       "10."
|     ]
|   }
| }
| )
| [
|   {
|     _id: ObjectId('69d7e9d56d510e0cfd8563b3'),
|     timestamp: ISODate('2025-03-04T09:00:00.000Z'),
|     rule_id: 'SID-2050',
|     rule_name: 'DNS Tunneling Detected',
|     src_ip: '10.0.2.100',
|     dst_ip: '8.8.8.8',
|     severity: 'critical',
|     action: 'block',
|     sensor: 'ids-internal-01'
|   }
| ]

```

FIGURE 8 – Requête du devoir maison : alerte critique provenant du réseau interne identifiée à l’aide de \$substrBytes.

5.3 Partie 3 — Index proposés et justifiés

Le support prévoit une troisième partie consacrée à la proposition de deux index. Or, **aucune commande `createIndex()` sur `ids_alerts` n’apparaît dans le journal capturé**. Par honnêteté méthodologique, les éléments ci-dessous sont donc présentés comme des **propositions argumentées** destinées à compléter le rendu, et non comme des opérations effectivement exécutées.

Index proposé	Justification
<code>{ rule_id: 1, timestamp: 1 }</code>	Accélère directement la requête sur SID-1001 triée par horodatage, qui est l’un des cas d’usage centraux du devoir. Il reste également cohérent pour des investigations chronologiques par signature IDS.
<code>{ timestamp: 1 }</code>	Couvre les recherches par fenêtre temporelle (08h–10h), ainsi que les requêtes <i>alerte la plus ancienne</i> et, par inversion de parcours, <i>alertes les plus récentes</i> . C’est l’index le plus universel pour un journal d’alertes horodaté.

Dans un contexte de volume beaucoup plus important, une refonte de certaines requêtes permettrait d’envisager d’autres index, par exemple sur `src_ip`. Toutefois, avec la contrainte de proposer seulement deux index et en restant fidèle aux requêtes effectivement observées, le couple ci-dessus est le plus défendable.

6 Checklist sécurité remplie à partir des preuves

La checklist du support a été complétée ci-dessous en distinguant ce qui est *effectivement démontré* dans les commandes et ce qui relève d’une *recommandation logique* issue du TP6.

Critère	Statut	Commentaire
Périmètre fonctionnel par collection	Fait	<code>users</code> pour les identités/équipes/rôles, <code>security_logs</code> pour les événements bruts, <code>incidents</code> pour la gestion de cas, <code>ids_alerts</code> pour le devoir maison.
PII identifiés	Fait	Les emails des utilisateurs sont explicitement présents dans <code>users</code> . Les journaux contiennent aussi des données opérationnelles sensibles via <code>raw_log</code> .
PII masqués pour accès non privilégiés	Fait	Le champ <code>email_masked</code> est généré pour les quatre utilisateurs.
Projections définies par rôle (viewer/analyst)	Partiel	La projection <code>viewer</code> est démontrée. L'équivalent <code>analyst</code> n'est pas formalisé par une commande dédiée dans le journal.
Champs haute sensibilité (raw_log)	Partiel	La nécessité de restreindre <code>raw_log</code> est démontrée par la requête d'exclusion ; l'affectation explicite à un rôle privilégié reste une recommandation de gouvernance.
Dénormalisation sans duplication de PII	Fait	Les incidents répliquent des identifiants et des tags, mais pas les emails des utilisateurs.
Index ne révélant pas de patterns sensibles	Fait	Les index créés portent sur <code>src_ip</code> et <code>timestamp/src_ip</code> ; aucun index sur <code>user_attempted</code> n'est observé.
Pas de sur-collecte	Partiel	La sur-collecte potentielle est identifiée via <code>raw_log</code> ; la réduction du schéma n'est pas exécutée, mais le besoin est correctement diagnostiqué.

7 Analyse critique et enseignements

Ce travail montre une progression technique crédible et cohérente. Trois enseignements majeurs se dégagent.

7.1 1. Embed/reference : compréhension correcte du compromis

La modélisation réalisée respecte bien le compromis enseigné : embarquer ce qui est borné et contextuel (commentaires, rôles), référencer ce qui peut croître sans limite ou doit être isolé (logs). Le fait d'avoir effectivement manipulé `related_log_ids` dans les incidents renforce la compréhension de ce choix.

7.2 2. L'indexation a été mesurée, pas supposée

La bascule de `COLLSCAN` vers `IXSCAN` prouve que l'étudiant a vérifié le comportement du moteur avec `explain()`. C'est beaucoup plus pertinent pédagogiquement que de se contenter de créer un index « au cas où ».

7.3 3. La sécurité data-level est comprise comme un problème d'exposition, pas seulement de stockage

Le masquage des emails, la projection `viewer` et l'exclusion de `raw_log` montrent que la réflexion ne porte pas uniquement sur *ce qui est stocké*, mais aussi sur *ce qui est rendu visible*. C'est exactement l'objectif du dernier bloc de la séance.

8 Conclusion

Au vu des preuves disponibles, le travail de séance peut être considéré comme **largement réalisé et techniquement satisfaisant**. Les TPs 1 à 6 apparaissent dans le journal avec des résultats cohérents, et le devoir maison a été exécuté sur toute sa partie opérationnelle de requêtage. Les erreurs initiales ont été corrigées proprement, les vérifications de cohérence ont été effectuées, et les sorties observées sont globalement en ligne avec les attentes du support.

Le seul élément qui n'est pas capturé comme action réalisée est la création explicite des deux index du devoir maison. Ce rapport compense ce manque en proposant deux index argumentés, tout en signalant clairement qu'il s'agit d'une complétion analytique et non d'une preuve d'exécution. Cette distinction est importante pour préserver la traçabilité du livrable.

A Extraits représentatifs des commandes exécutées

A.1 Erreur initiale puis correction

```
// tentative initiale
let charlieLogIds = db.security_logs.find({
  src_ip: "10.0.1.200",
  timestamp: { $gte: new Date("2025-03-03T17:00:00Z") }
}).map(d => d._id)

db.incidents.insertOne({
  incident_id: "INC-2025-003",
  related_log_ids: charlieLogIds,
  comments: []
})
// BSONError: Cannot convert circular structure to BSON
```

```
// version stabilisée
let charlieLogIds = db.security_logs
  .find({
    src_ip: "10.0.1.200",
    timestamp: { $gte: new Date("2025-03-03T17:00:00Z") }
  })
  .toArray()
  .map(d => d._id);

db.incidents.insertOne({
  incident_id: "INC-2025-003",
  related_log_ids: charlieLogIds,
  comments: []
});
```


A.2 Séquence d'indexation

```
db.security_logs.find({ src_ip: "192.168.1.105" }).explain("executionStats")
db.security_logs.createIndex({ src_ip: 1 })
db.security_logs.find({ src_ip: "192.168.1.105" }).explain("executionStats")
db.security_logs.createIndex({ timestamp: 1, src_ip: 1 })
db.security_logs.getIndexes()
```

A.3 Masquage des emails

```
db.users.updateMany({}, [
  { $set: { email_masked: {
    $concat: [
      { $substr: ["$email", 0, 1] },
      "***@",
      { $arrayElemAt: [{ $split: ["$email", "@"] }, 1] }
    ]
  }}}
])
```

A.4 Extrait du devoir maison

```
db.ids_alerts.find(
  {
    severity: "critical",
    $expr: {
      $eq: [
        { $substrBytes: ["$src_ip", 0, 3] },
        "10."
      ]
    }
  }
)
```

B Inventaire condensé des séquences du journal

#	Phase	Action documentée	Statut
1	Pré-tests / erreurs initiales	Première tentative d'insertion des incidents avec récupération directe des ObjectId via <code>.map()</code> ; échec.	Erreur
2	Pré-tests / erreurs initiales	Première tentative de création de l'incident INC-2025-003 avec récupération directe des ObjectId ; échec.	Erreur
3	TP1 — Modèle SOC-lite	Insertion de la collection users.	OK
4	TP1 — Modèle SOC-lite	Insertion initiale de la collection security_logs.	OK
5	TP1 — Modèle SOC-lite	Matérialisation de logIds via <code>toArray().map()</code> .	OK
6	TP1 — Modèle SOC-lite	Insertion correcte des incidents INC-2025-001 et INC-2025-002.	OK

#	Phase	Action documentée	Statut
7	TP1 — Modèle SOC-lite	Comptage des utilisateurs.	OK
8	TP1 — Modèle SOC-lite	Comptage des logs de sécurité.	OK
9	TP1 — Modèle SOC-lite	Comptage des incidents.	OK
10	TP1 — Modèle SOC-lite	Lecture de contrôle de INC-2025-001.	OK
11	TP2 — Incidents et références	Ajout d'un commentaire embarqué dans INC-2025-001.	OK
12	TP2 — Incidents et références	Projection des commentaires de INC-2025-001.	OK
13	TP2 — Incidents et références	Insertion de trois nouveaux logs liés à Charlie.	OK
14	TP2 — Incidents et références	Création correcte de INC-2025-003 avec <code>related_log_ids</code> matérialisés.	OK
15	TP2 — Incidents et références	Vérification du nombre de logs liés à INC-2025-003.	OK
16	TP2 — Incidents et références	Ajout d'analystes assignés à INC-2025-002.	OK
17	TP3 — Requêtes guidées	Recherche des logs critiques.	OK
18	TP3 — Requêtes guidées	Recherche par fenêtre temporelle 10h–12h, tri croissant.	OK
19	TP3 — Requêtes guidées	Projection minimale sur les échecs de connexion.	OK
20	TP3 — Requêtes guidées	Récupération des trois logs les plus récents.	OK
21	TP3 — Requêtes guidées	Filtre logique sur <code>src_ip</code> ou sévérité critique.	OK
22	TP3 — Requêtes guidées	Lecture des incidents actifs/en investigation avec commentaires.	OK
23	TP3 — Requêtes guidées	Recherche des événements du device <code>siem-console</code> .	OK
24	TP3 — Requêtes guidées	Incidents assignés à <code>alice_analyst</code> .	OK
25	TP4 — Agrégations	Top 5 des IP sources par agrégation.	OK
26	TP4 — Agrégations	Échecs de connexion par utilisateur.	OK
27	TP4 — Agrégations	Répartition des sévérités.	OK
28	TP4 — Agrégations	Alertes high/critical par device.	OK
29	TP4 — Agrégations	Nombre d'événements par heure.	OK
30	TP5 — Index et <code>explain()</code>	Explains sans index sur <code>src_ip</code> = COLLSCAN.	OK
31	TP5 — Index et <code>explain()</code>	Création de l'index simple sur <code>src_ip</code> .	OK
32	TP5 — Index et <code>explain()</code>	Explains après index = IXSCAN.	OK
33	TP5 — Index et <code>explain()</code>	Création de l'index composé <code>{timestamp, src_ip}</code> .	OK

#	Phase	Action documentée	Statut
34	TP5 — Index et explain()	Explain sur requête temporelle + IP.	OK
35	TP5 — Index et explain()	Liste des index présents.	OK
36	TP5 — Index et explain()	Explain illustrant la prefix rule / choix du planificateur.	OK
37	TP6 — Sécurité data-level	Lecture d'un log malware pour identifier les champs sensibles.	OK
38	TP6 — Sécurité data-level	Masquage des emails par update pipeline.	OK
39	TP6 — Sécurité data-level	Contrôle du masquage email_masked.	OK
40	TP6 — Sécurité data-level	Projection sûre pour un viewer sur users.	OK
41	TP6 — Sécurité data-level	Exclusion de raw_log pour les viewers.	OK
42	Devoir maison — IDS alerts	Insertion du mini jeu de données ids_alerts.	OK
43	Devoir maison — IDS alerts	Filtre severity high/critical.	OK
44	Devoir maison — IDS alerts	Recherche SID-1001 triée par timestamp.	OK
45	Devoir maison — IDS alerts	Comptage des alertes par rule_id.	OK
46	Devoir maison — IDS alerts	Fenêtre 08h–10h avec projection minimale.	OK
47	Devoir maison — IDS alerts	Deux alertes les plus récentes.	OK
48	Devoir maison — IDS alerts	Top IPs sources du devoir.	OK
49	Devoir maison — IDS alerts	Alertes dont action = block ou drop.	OK
50	Devoir maison — IDS alerts	Nombre d'alertes par sensor.	OK
51	Devoir maison — IDS alerts	Alertes critiques du réseau interne via \$substrBytes.	OK
52	Devoir maison — IDS alerts	Alerte la plus ancienne.	OK